

Express.js: Day 3

Routing, multiple routes, HTTP methods, and route matching.

Day 3 goal

Understand Express routing, register multiple routes, understand the main HTTP methods, and understand that route matching depends on both HTTP method and path.

What you learned today

Topic	Core idea
Routing	Defines how the application responds to particular HTTP methods and paths.
Multiple routes	Different paths can have different handlers.
HTTP methods	GET, POST, PUT, PATCH and DELETE can be registered as different routes.
Route matching	The incoming method and path must match the registered route for its handler to execute.
Same path, different method	GET /users and POST /users are different routes because their methods differ.

1. What is Routing?

Routing means defining how an Express application responds when an incoming request matches a particular HTTP method and path.

```
app.get("/", (req, res) => {
  res.send("Home");
});

app.get("/about", (req, res) => {
  res.send("About");
});

app.get("/contact", (req, res) => {
  res.send("Contact");
});
```

These are three separate GET routes: GET /, GET /about, and GET /contact.

2. Multiple Routes

An Express application can register many routes. Each route can have its own path and handler.

```
app.get("/", (req, res) => {
  res.send("Welcome to my website");
});

app.get("/about", (req, res) => {
  res.send("About Bhavana");
});
```

```
app.get("/contact", (req, res) => {  
  res.send("Contact Bhavana");  
});
```

Mental model: Express application → incoming request → find matching method + path → execute handler → send response.

3. Main HTTP Methods

Express provides methods such as `app.get()`, `app.post()`, `app.put()`, `app.patch()`, and `app.delete()` for registering routes for those HTTP methods.

```
app.get("/users", (req, res) => {  
  res.send("Get users");  
});  
  
app.post("/users", (req, res) => {  
  res.send("Create user");  
});  
  
app.put("/users", (req, res) => {  
  res.send("Update user");  
});  
  
app.patch("/users", (req, res) => {  
  res.send("Partially update user");  
});  
  
app.delete("/users", (req, res) => {  
  res.send("Delete user");  
});
```

4. Basic Purpose of Each Method

Method	Basic purpose
GET	Retrieve/read information.
POST	Create/add a resource.
PUT	Update/replace a resource representation.
PATCH	Partially modify a resource.
DELETE	Remove/delete a resource.

Important: These are the basic purposes for Day 3. Deeper REST semantics, CRUD, API design, and Postman testing belong to the separate REST API phase.

5. Same Path, Different HTTP Methods

The same path can be registered with different HTTP methods. They are different routes because route matching considers the method as well as the path.

```
app.get("/users", (req, res) => {
  res.send("GET users");
});

app.post("/users", (req, res) => {
  res.send("POST users");
});

GET /users → first handler
POST /users → second handler
```

Key rule: Route matching = HTTP method + path. The path alone is not enough.

6. Why the Browser Hits app.get()

When you type a normal URL into a browser and navigate to it, the browser normally makes a GET request.

```
http://localhost:3000/users
    ↓
GET /users
```

Therefore, if both GET /users and POST /users exist, normal browser navigation to /users matches the GET route.

Testing note: POST, PUT, PATCH, and DELETE are better tested with an API client such as Postman when we reach the REST API portion.

7. PUT vs PATCH

PUT is generally associated with updating/replacing a resource representation. PATCH is intended for partial modification.

```
PUT /users/10
{
  "name": "Bhavana",
  "age": 27,
  "city": "Mysore"
}

PATCH /users/10
{
  "city": "Mysore"
}
```

Simple mental model:
PUT → "Make the resource become this representation."
PATCH → "Modify this part of the resource."

An API implementation can choose its own behavior, but the standard HTTP semantics distinguish PUT from PATCH in this way. We will revisit deeper REST semantics later.

8. DELETE

DELETE is generally used to remove a resource.

```
app.delete("/users", (req, res) => {  
  res.send("User deleted");  
});
```

A real API will normally identify the resource more specifically, often with a route parameter such as `/users/10`. Route parameters are a later Express topic.

9. Complete Day 3 Example

```
const express = require("express");

const app = express();

app.get("/", (req, res) => {
  res.send("Welcome to my website");
});

app.get("/about", (req, res) => {
  res.send("About Bhavana");
});

app.get("/contact", (req, res) => {
  res.send("Contact Bhavana");
});

app.get("/users", (req, res) => {
  res.send("Here is the user");
});

app.post("/users", (req, res) => {
  res.send("User Created");
});

app.put("/users", (req, res) => {
  res.send("Update user");
});

app.patch("/users", (req, res) => {
  res.send("Partially update user");
});

app.delete("/users", (req, res) => {
  res.send("Delete user");
});

app.listen(3000);
```

10. Common Beginner Mistakes

Mistake	Reality
Thinking /users alone determines the route	The HTTP method matters too.
Thinking GET and POST /users are the same route	They are different routes because the methods differ.
Testing POST by typing the URL in Chrome	Normal browser navigation sends GET; use an API client for other methods.
Treating PUT and PATCH as identical	They have different intended HTTP semantics; deeper REST behavior comes later.
Putting REST API design inside Express fundamentals	Our roadmap keeps REST APIs as a separate 8-day phase.

11. Day 3 Assessment — Reference Answers

1. Routing is defining how an Express application responds when a particular HTTP method and path are matched.
2. GET /users and POST /users are different routes because their HTTP methods differ even though the path is the same.
3. A normal browser visit to /users normally executes the GET route because the browser sends GET.
4. GET → retrieve/read; POST → create/add; PUT → update/replace; PATCH → partially modify; DELETE → remove.
5. PUT /users will not execute either a GET /users or POST /users handler because no registered route matches both the method and path.

12. Day 3 Quick Cheat Sheet

ROUTING

HTTP method + path → matching route handler

GET	→ retrieve/read
POST	→ create/add
PUT	→ update/replace
PATCH	→ partial modification
DELETE	→ remove/delete

Same path + different method = different route

13. Rebuild Day 3 Years Later

Remember:

1. A route connects an HTTP method + path to a handler.
2. Express can have many routes.
3. The same path can be registered for different methods.
4. Both method and path must match for the handler to execute.
5. GET/POST/PUT/PATCH/DELETE represent different kinds of HTTP operations.
6. REST API design and deeper method semantics are learned separately.

Day 3 status

EXPRESS.JS DAY 3 — COMPLETE ☑

You built and tested multiple routes, registered multiple HTTP methods, explained route matching, and passed the final Day 3 assessment.